Operating Systems Lab — Assignment 8 Report: Memory Management in xv6

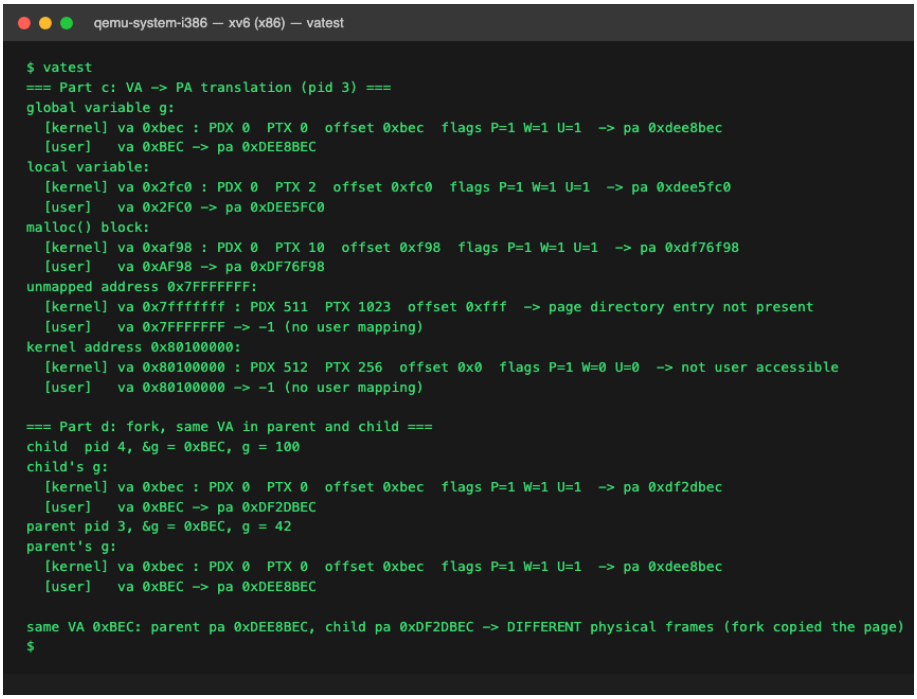
Manav Singla, 2402MC02 — xv6 (x86, mit-pdos/xv6-public), qemu-system-i386, i686-elf-gcc 16 — Questions 1–4

Q1 — Virtual to physical address translation: `va2pa(int va)`

- `walkpgdir()` in `vm.c` was made non-static and declared in `defs.h`. New kernel function `va2pa(pgdir, va)` splits the 32-bit VA into `PDX(va)` (bits 31–22), `PTX(va)` (bits 21–12) and offset (bits 11–0), checks the PDE, walks to the PTE, and returns `PTE_ADDR(*pte) | offset`. It returns −1 if the PDE or PTE is not present, or if `PTE_U` is clear.
- The kernel side prints PDX, PTX, offset and the P/W/U bits with `cprintf`, and the user side prints the result.
- `vatest.c` results:

Object	VA	PDX / PTX / offset	Flags	PA
global `g`	0xBEC	0 / 0 / 0xBEC	P W U	0xDEE8BEC
local variable (stack)	0x2FC0	0 / 2 / 0xFC0	P W U	0xDEE5FC0
malloc() block (heap)	0xAF98	0 / 10 / 0xF98	P W U	0xDF76F98
0x7FFFFFFF	0x7FFFFFFF	511 / 1023 / 0xFFF	PDE not present	−1
kernel 0x80100000	0x80100000	512 / 256 / 0x0	P, U clear	−1
`g` in parent / child after fork	0xBEC	0 / 0 / 0xBEC	P W U	0xDEE8BEC / **0xDF2DBEC**

Parent and child use the same virtual address but different physical frames, because `copyuvm()` gives the child a private copy of every page. The child also changed `g` to 100 while the parent still sees 42.



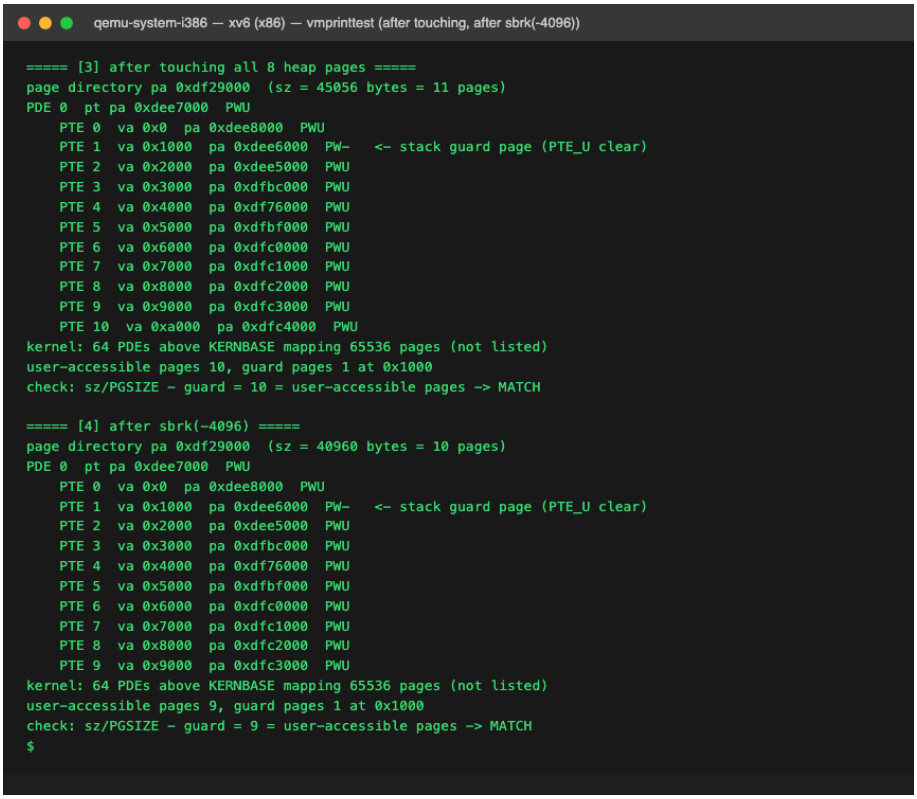
Q2 — Physical memory statistics: `getmeminfo(struct meminfo *mi)`

- `kmem` in `kalloc.c` gains `nfree` (incremented in `kfree()`, decremented in `kalloc()`, both under `kmem.lock`) and `ntotal` (counted once in `freerange()` during boot). `kmemstat()` reads both under the lock.
- New shared header `meminfo.h`: `struct meminfo { total_pages; free_pages; proc_pages; pt_pages; }`. `proc_pages = sz / PGSIZE`; `pt_pages` = 1 page directory + every present PDE (user and kernel page tables), computed by `ptpages()` in `vm.c`.

memtest step	free pages	change	proc_pages
start	56789	–	3
sbrk(64 pages) and touch each	56725	−64	67
pipe created (before fork)	56724	−1 (pipe buffer)	67
child alive	56590	−134 = 67 user + 66 page-table + 1 kernel stack	67
child exited and reaped	**56724**	+134 → equals before fork: **no leak**	67
sbrk(−64 pages), pipe closed	**56789**	back to the starting value	3

Q3 — Dumping the page table: `vmprint()`

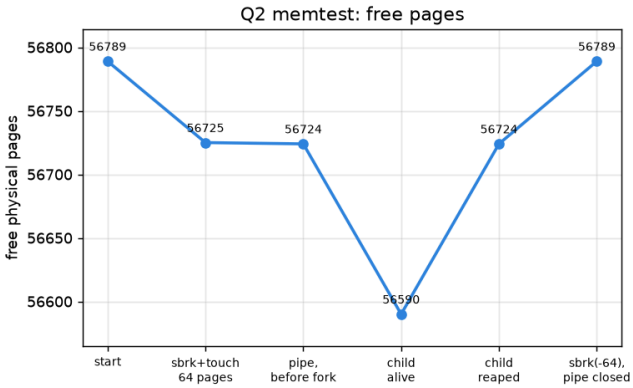
- `vmprint(pgdir, sz)` loops over the 1024 PDEs. For each present PDE below `KERNBASE` it prints the index, the page table's physical address and flags, then each present PTE indented with entry index, VA (`PGADDR(pdx, ptx, 0)`), frame PA and P/W/U. PDEs at or above `KERNBASE` are only counted, giving a one-line summary (64 PDEs mapping 65,536 kernel pages).
- A present page with `PTE_U` clear is labelled as the **stack guard page**. exec() creates it with `clearpteu(sz - 2*PGSIZE)` ; here it is at VA 0x1000, between the code/data page (0x0) and the stack page (0x2000).
- Check printed by every dump: user-accessible pages = `sz/PGSIZE - guard` . At start-up: 3 pages – 1 guard = 2 → MATCH. After `sbrk(8*4096)` , sz is 11 pages but only 2 are mapped, because the heap is lazy (Q4); the dump reports "8 heap pages not yet touched". After touching them, PTEs 3–10 appear and the check is MATCH (10). After `sbrk(-4096)` , PTE 10 disappears and the check is MATCH (9).

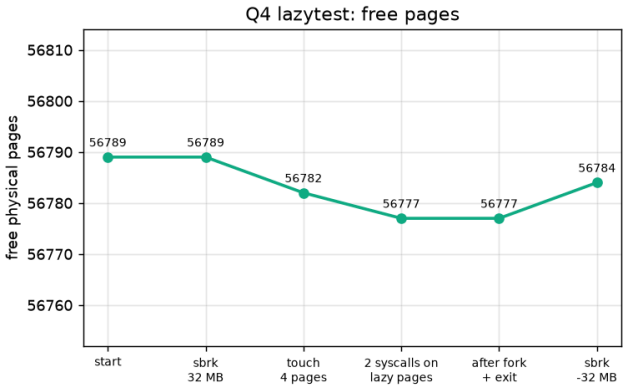


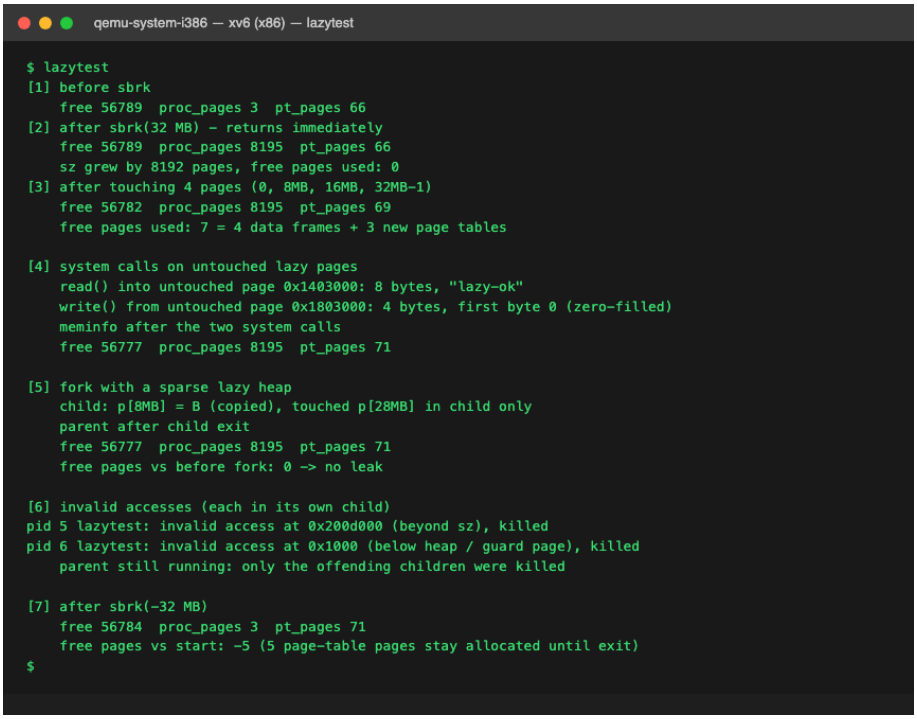
Q4 — Lazy allocation of the heap

- **sys_sbrk:** for `n > 0` it only checks the new size against `KERNBASE` (and wrap-around) and raises `sz` . For `n < 0` it still calls `growproc()` , so `deallocuvm()` frees only the pages that are really present.
- **Heap boundary:** new field `proc->heapstart` , set by exec() to the end of the user stack and copied by fork(). The heap is exactly `[heapstart, sz)` , which lies above the guard and stack pages.
- **T_PGFLT handler (trap.c):** reads `rcr2()` . If `heapstart ≤ va < sz` , `lazyalloc()` takes a frame from `kalloc()` , zeroes it and maps it with `PTE_W|PTE_U` , and the faulting instruction is retried. If `kalloc()` fails, the process is killed. A user fault outside the heap (beyond sz, or on the guard page or code/stack) prints a message and kills only that process.
- **Kernel touching lazy memory:** the same handler runs for faults taken in kernel mode, so `read()` into an untouched buffer (the kernel copies straight to the user VA) and `write()` from one both work. Other kernel faults still panic. `copyuvm()` now skips non-present PTEs and missing page tables instead of panicking, so fork works with a sparse heap.

lazytest step	free pages	change	proc_pages
before sbrk	56789	–	3
sbrk(32 MB) returns immediately	56789	**0**	8195
touch 0, 8 MB, 16 MB, 32 MB–1	56782	−7 = 4 frames + 3 new page tables	8195
read()/write() on 2 untouched pages	56777	−5 = 2 frames + 2 page tables + 1 pipe	8195
fork, child touches 28 MB, exits	56777	0 (no leak)	8195
access sz + 40 KB / guard page	each child killed ("invalid access … killed"); parent and kernel keep running		
sbrk(−32 MB)	56784	−5 vs start = page tables kept until exit	3







Problems faced and how they were resolved

- **Toolchain.** The x86 xv6 targets i386, and macOS has no such compiler by default. I installed `i686-elf-gcc` (GCC 16). Its newer warnings (`array-bounds` on `mp.c` 's BIOS pointer scan, `unused-but-set-variable` in usertests) break the `-Werror` build, so those warnings are disabled in the Makefile and `-std=gnu11` is set. No kernel logic changed for this.
- **`pte_t` in defs.h.** `pte_t` is defined in `mmu.h` , which most files including `defs.h` don't include, so exporting `walkpgdir` broke compilation. The prototype uses `uint*` , the same type.
- **Defining "inside the heap".** `sz` alone cannot separate the heap from the guard page, so faults on the guard page would have been "fixed" by mapping memory over it. The new `heapstart` field makes the guard page and stack fall outside the lazy range, so touching the guard page kills the process as expected.
- **Interaction with Q2/Q3.** With lazy sbrk, growing the heap no longer lowers free pages, so `memtest` and `vmprinttest` touch each new page, and `vmprint` reports untouched heap pages explicitly instead of a false mismatch.
- **Free pages not returning after shrink.** `deallocuvm()` frees data frames but keeps the page-table pages, so after `sbrk(-32 MB)` free pages are 5 below the start. These pages are released by `freevm()` at exit, which the fork/exit leak test confirms (0 difference). An early run also showed −6 because the test's pipe was still open; closing it gives exactly the page-table count.
- **Regression check.** The stock `usertests` still prints ALL TESTS PASSED with lazy allocation enabled.